

# Local-First Implementation Blueprint

A complete execution plan for a React + Vite and FastAPI application that collects made-to-measure data, produces transparent outfit recommendations, and is designed to scale to 100,000 participants.

| Decision              | Selected approach                                                             |
|-----------------------|-------------------------------------------------------------------------------|
| Development mode      | Local only; deployment and hosting deliberately excluded                      |
| Frontend              | React JavaScript + Vite                                                       |
| Backend               | FastAPI + Pydantic + SQLAlchemy                                               |
| Database              | Local PostgreSQL from the beginning                                           |
| Recommendation method | Deterministic, versioned rule engine before any machine learning              |
| Data target           | 100,000 participants; roughly 2-3 million measurement values                  |
| Safety boundary       | No real child-data collection before valid guardian consent and expert review |

Prepared 4 September 2026 | Target environment: Fedora 42, 16 GB RAM, RTX 3060 6 GB. The GPU is not required for the initial system.

SECTION 00

# Contents and document use

| Part | Topic                                          |
|------|------------------------------------------------|
| 01   | Executive decisions and success definition     |
| 02   | Product scope, users and boundaries            |
| 03   | Measurement framework                          |
| 04   | 100,000-participant collection strategy        |
| 05   | Consent, privacy and child-data safeguards     |
| 06   | System architecture and technology choices     |
| 07   | Database and information model                 |
| 08   | Recommendation and fit-calculation engine      |
| 09   | Frontend journey and interface behavior        |
| 10   | Backend modules and API contract               |
| 11   | Local project structure and setup              |
| 12   | Testing, load simulation and quality gates     |
| 13   | Sampling, analytics and future model readiness |
| 14   | Implementation roadmap                         |
| 15   | Risk register and mitigations                  |
| 16   | Definition of done and launch gates            |
| A    | Appendix: API examples and commands            |
| B    | Appendix: source references                    |

How to use this document: implement sections 06-12 in the order defined by section 14. Do not treat the example ease allowances or demographic bands as validated apparel standards; they remain configuration inputs that require domain review.

## SECTION 01

# Executive decisions and success definition

The platform should be built as a made-to-measure recommendation system. It will collect body measurements and style preferences, validate data quality, choose compatible garment designs, apply configurable ease rules, rank outfit combinations, and produce a tailor-facing specification. It must not claim to generate a production-ready sewing pattern in version 1.

## What the first complete version does

- 1 Select a garment combination: shirt and trousers, T-shirt and trousers, or one garment.
- 2 Collect only the measurements required for the selected garments.
- 3 Collect occasion, climate, fit, style, colour and fabric preferences.
- 4 Normalize units, evaluate measurement quality and surface understandable warnings.
- 5 Return three deterministic, ranked outfit recommendations with score explanations.
- 6 Display both submitted body measurements and calculated finished-garment targets.
- 7 Save the measurement session, consent status, rules version and recommendation snapshot.

## Non-negotiable architecture decisions

| Area                | Decision                             | Reason                                                                       |
|---------------------|--------------------------------------|------------------------------------------------------------------------------|
| Database            | PostgreSQL                           | Supports concurrent writes, constraints, analytics and migration discipline. |
| Fit logic           | Versioned rules                      | Easy to test, explain and revise with a tailor.                              |
| Measurement storage | Integer millimetres                  | Avoids unit drift and floating-point comparison errors.                      |
| Forms               | Backend-driven schema                | Prevents React and Python field definitions from diverging.                  |
| Data rights         | Consent and deletion are first-class | They cannot be retrofitted safely after large-scale collection.              |
| Testing             | Synthetic scale before real data     | Exposes data, transaction and performance defects without privacy risk.      |

## Success at local-development completion

- A fresh local installation can run the frontend, backend and PostgreSQL using documented commands.
- One hundred thousand synthetic participants can be inserted in batches without duplicates or corruption.
- The same accepted input always produces the same recommendation for a fixed rules and catalog version.
- No external API, cloud database, authentication provider or GPU is required.
- Adult, guardian and data-withdrawal workflows are represented and tested, even though real-world legal verification is deferred.

## SECTION 02

# Product scope, users and boundaries

## Primary user journeys

| User                       | Goal                             | Required flow                                                                 |
|----------------------------|----------------------------------|-------------------------------------------------------------------------------|
| Adult participant          | Receive a tailored outfit        | Consent -> garment choice -> measurements -> preferences -> results.          |
| Parent or guardian         | Enter a child's measurements     | Age gate -> guardian verification -> child notice -> measurements -> results. |
| Tailor or trained measurer | Record reliable measurements     | Participant code -> protocol -> repeat measurements -> review.                |
| Research administrator     | Monitor quality, not individuals | Aggregate dashboard -> flagged sessions -> controlled export.                 |
| Developer                  | Test scale locally               | Synthetic generator -> migrations -> load tests -> backup and restore.        |

## Version 1 boundaries

| Included now                     | Explicitly deferred                                         |
|----------------------------------|-------------------------------------------------------------|
| Shirts, T-shirts and trousers    | Jackets, dresses, footwear and complex traditional garments |
| Manual tape measurements         | Camera estimation, body scanning and virtual try-on         |
| Local garment images and catalog | Marketplace inventory and shopping integrations             |
| Rules-based ranking              | LLM recommendations and machine-learning sizing             |
| Local PostgreSQL                 | Hosting, cloud networking and production observability      |
| Tailor-facing specification      | Automated sewing pattern or cutting file                    |

Production-pattern warning: finished garment dimensions are not a sewing pattern. Pattern drafting also depends on seam allowances, fabric stretch and recovery, posture, darts, construction method, style lines and fitting adjustments. That work requires an apparel-pattern specialist.

## Demographics: collect only what changes the outcome

Age, sex at birth, gender identity, garment presentation and measurements are different concepts. The engine must primarily use actual measurements and the chosen garment cut. Sex at birth should be optional unless a documented pattern or research requirement justifies it. Gender identity should never be used as a proxy for body shape. Sexual orientation has no tailoring purpose and must not be collected.

SECTION 03

# Measurement framework

Source-aligned fields, unambiguous internal codes and quality metadata.

## Garment-specific measurement sets

| Garment  | Required                                                                                 | Optional for higher precision                           |
|----------|------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Shirt    | Neck, chest/bust, waist, hip, shoulder width, sleeve length, armhole depth, shirt length | Back width, front length, wrist cuff, yoke depth, bicep |
| Trousers | Waist, hip, inseam, outseam, thigh, knee, leg opening                                    | Total rise, front rise, back rise, seated crotch depth  |
| T-shirt  | Chest/bust, waist, hip, shoulder width, sleeve length, armhole depth, T-shirt length     | Back width, neck, bicep                                 |

Use distinct codes such as `wrist_cuff_circumference` and `leg_opening_circumference`. The source guide uses cuff terminology in more than one garment context; ambiguous shared names will create defects.

## Measurement-definition record

Each field is defined once in a backend-owned, versioned catalog and exposed through the measurement-schema API. The React wizard renders from this response.

```
{
  "code": "chest_circumference",
  "label": "Chest/Bust Circumference",
  "kind": "circumference",
  "required_for": ["shirt", "tshirt"],
  "instruction": "Measure around the fullest part with the tape level.",
  "repeat_required": true,
  "protocol_version": 1
}
```

## Measurement protocol

- 1 Show a diagram, tape-placement instruction and posture instruction before the first value.
- 2 Record the entered unit, but immediately normalize accepted values to integer millimetres.
- 3 Measure high-impact values twice. If the difference exceeds the configured tolerance, request a third attempt.
- 4 Store every attempt and use the median accepted value; never overwrite the first attempt.
- 5 Attach measurer type, method, protocol version, confidence and validation status to the session.
- 6 Flag unusual measurements for review instead of automatically deleting them.

## Three validation layers

| Layer             | Examples                                                              | Typical result                        |
|-------------------|-----------------------------------------------------------------------|---------------------------------------|
| Technical         | Missing, negative, NaN, infinity, unsupported code, oversized payload | Reject with a structured 422 response |
| Cross-measurement | Inseam greater than outseam; inconsistent rise components             | Warning or third measurement request  |
| Age-aware         | Value outside broad range for the participant's age band              | Flag for confirmation; do not erase   |

## SECTION 04

# 100,000-participant collection strategy

At this scale the main risk is not raw database capacity. It is collecting inconsistent, biased, duplicated or unlawfully processed information. The platform must therefore treat consent, provenance, retries, quality and withdrawal as core data rather than secondary features.

## Expected local scale

| Entity               | Planning quantity | Notes                                        |
|----------------------|-------------------|----------------------------------------------|
| Participants         | 100,000           | Pseudonymous participant records             |
| Measurement sessions | 100,000-150,000   | Supports repeated measurement over time      |
| Measurement values   | 2-3 million       | Includes repeat attempts                     |
| Consent records      | 100,000+          | Multiple purposes and versions may exist     |
| Recommendation runs  | 100,000+          | Input and rule snapshots for reproducibility |

## Submission reliability

- Every mutating request includes an Idempotency-Key; a retry returns the original result.
- Database uniqueness constraints protect against double clicks and network retries.
- A measurement submission and its status update occur in one transaction.
- Draft sessions support autosave; final submission is a separate irreversible state transition.
- Raw payloads are preserved for investigation, while normalized values live in structured tables.
- List endpoints use cursor pagination; exports stream in batches and never load the full dataset into API memory.

## Progressive collection plan

| Gate        | Population        | Purpose                                    | Exit requirement                                |
|-------------|-------------------|--------------------------------------------|-------------------------------------------------|
| Synthetic   | 100,000 generated | Schema, constraints, performance           | No corruption, stable memory, reproducible load |
| Internal    | 20-50 adults      | Find instruction and usability errors      | Critical form defects resolved                  |
| Pilot       | 100-500 consented | Compare self and professional measurements | Quality thresholds validated                    |
| Expansion 1 | 1,000             | Sampling and operations                    | Quota gaps and support burden understood        |
| Expansion 2 | 10,000            | Reliability and fairness review            | Independent review and deletion workflow proven |
| Target      | 100,000           | Full study or service collection           | All earlier gates documented and approved       |

Local-only limitation: the code can be built and load-tested locally, and existing CSV data can be imported. Actual remote collection from 100,000 people will eventually require an accessible deployed service. Hosting remains outside this plan.

SECTION 05

# Consent, privacy and child-data safeguards

## Privacy-by-design requirements

- Use a random public participant code rather than a person's name as the operational identifier.
- Avoid email, telephone, precise address and exact date of birth unless a specific workflow needs them.
- Record age in completed months at measurement time; show years in the interface for adults.
- Separate optional research consent from the consent needed to generate the requested recommendation.
- Make withdrawal and deletion as easy to find as consent.
- Do not log measurement payloads, consent evidence, contact information or guardian identifiers.
- Use a data-retention policy defined before collection; do not retain child data indefinitely by default.

## Child workflow

| Step                 | System behavior                                                                          |
|----------------------|------------------------------------------------------------------------------------------|
| 1. Age gate          | Determine whether the applicable jurisdiction requires guardian consent.                 |
| 2. Guardian notice   | Describe the fields, purpose, retention and deletion rights in plain language.           |
| 3. Verification      | Use an approved method that reasonably verifies the consenting adult or lawful guardian. |
| 4. Child explanation | Provide an age-appropriate explanation and obtain assent where appropriate.              |
| 5. Collection        | Collect only necessary measurements; disable advertising and behavioral monitoring.      |
| 6. Rights            | Provide guardian access, correction, consent withdrawal and erasure pathways.            |

India-specific finding: the Digital Personal Data Protection Act defines a child as an individual under 18 and provides for verifiable parental consent before processing a child's personal data. It also restricts tracking, behavioral monitoring and targeted advertising directed at children. The 2025 Rules describe due diligence for verifying the parent. These requirements must be checked again before real collection because commencement and implementation timelines can change.

## Consent purposes

| Purpose                                      | Default              | Can be withdrawn?                   |
|----------------------------------------------|----------------------|-------------------------------------|
| Generate the requested outfit recommendation | Required for service | Yes; consequences clearly explained |
| Save measurement profile                     | Off until selected   | Yes                                 |
| Use de-identified data for research          | Off                  | Yes                                 |
| Use data to improve fit rules or models      | Off                  | Yes                                 |
| Marketing or advertising                     | Not included         | Not part of the product scope       |

This document is a technical plan, not legal advice. A privacy professional must validate consent language, verification mechanisms, retention rules and applicable jurisdictions before real-world child collection.

## SECTION 06

# System architecture and technology choices

## Local architecture

| Layer           | Component               | Responsibility                                                         |
|-----------------|-------------------------|------------------------------------------------------------------------|
| Presentation    | React + Vite            | Consent and measurement wizard, preferences, results, profile controls |
| API             | FastAPI + Pydantic      | Request validation, routes, error contracts and OpenAPI documentation  |
| Domain          | Python services         | Consent state machine, measurement quality, fit calculation, ranking   |
| Persistence     | SQLAlchemy + Alembic    | Transactions, models, constraints, migrations and repositories         |
| Database        | Local PostgreSQL        | Participants, sessions, values, consent, catalog and result snapshots  |
| Offline tooling | Python scripts + Locust | Synthetic data, imports, exports, statistics and load testing          |

## Frontend dependencies

- React Router for route-level flow.
- React Hook Form for wizard form state and field errors.
- TanStack Query for server state, retries and cache invalidation.
- CSS Modules or plain CSS for predictable styling.
- Vitest, React Testing Library and one browser-level tool for tests.

## Backend dependencies

- FastAPI, Uvicorn and Pydantic Settings.
- SQLAlchemy 2, Psycopg 3 and Alembic.
- Pytest and HTTPX for unit and API tests.
- Faker only for synthetic test records; do not create fake contact information unless a test specifically needs it.
- Locust for local concurrency and retry simulations.

## Complexity deliberately rejected

Do not add Docker, Redis, Celery, Kafka, microservices, sharding, an LLM, a vector database or a second database during the local MVP. At 100,000 participants, correct schema design, transactions, indexes and batch processing are sufficient.

## SECTION 07

# Database and information model

## Core entities

| Entity               | Essential fields                                                            | Key rule                                              |
|----------------------|-----------------------------------------------------------------------------|-------------------------------------------------------|
| participants         | id UUID, public_code, broad region, status, timestamps                      | No direct identifiers by default                      |
| demographic_profiles | participant_id, sex_at_birth, gender_identity, cut preference               | Optional sensitive fields remain separate             |
| measurement_sessions | participant_id, age_months, method, unit, schema/protocol versions, quality | One person may have many sessions                     |
| measurement_values   | session_id, code, value_mm, attempt, confidence, status                     | Unique attempt per measurement code                   |
| consent_records      | participant_id, version, purpose, granted/withdrawn timestamps              | One record per purpose and version                    |
| guardian_records     | verification state, method and timestamp                                    | Identifying evidence isolated                         |
| raw_submissions      | payload, received_at, idempotency key, processing status                    | Immutable troubleshooting record subject to retention |
| recommendation_runs  | session, rule/catalog versions, input snapshot, scores, results             | Reproducible historical recommendation                |
| audit_events         | actor type, action, entity reference, timestamp                             | No raw body measurements in audit logs                |

## Indexes and constraints

- Unique index on participants.public\_code and raw\_submissions.idempotency\_key.
- Unique constraint on measurement\_values(session\_id, measurement\_code, attempt\_number).
- Composite index on measurement\_sessions(participant\_id, created\_at).
- Operational indexes on measurement session status and quality score.
- Analysis index on measurement\_values(measurement\_code, value\_mm).
- Consent index on consent\_records(participant\_id, purpose, granted).
- Foreign keys with explicit deletion behavior; no accidental cascade across research records.

## Data-state model

Measurement sessions transition through draft -> submitted -> validated -> accepted or flagged. Withdrawal and erasure are separate rights-handling states. A final submission cannot silently return to draft; corrections create a revised session or an auditable revision.

Do not partition the database at this scale. Two to three million measurement rows are routine for PostgreSQL. Only add partitioning after measured query evidence demonstrates a real need.

SECTION 08

# Recommendation and fit-calculation engine

## Processing pipeline

- 1 Authorize processing based on the recorded consent purpose.
- 2 Normalize entered units to integer millimetres and retain the entered unit for display.
- 3 Run technical, cross-measurement and age-aware validation.
- 4 Select the garment-specific and age-appropriate fit rule set.
- 5 Filter garment templates by category, exclusions, occasion and required measurements.
- 6 Calculate target finished measurements using versioned ease allowances.
- 7 Score compatible top and bottom combinations, then apply deterministic tie-breaking.
- 8 Return three results with reasons, warnings, score breakdown and version metadata.
- 9 Persist an input and result snapshot for reproducibility.

## Fit formula

```
finished_measurement_mm = body_measurement_mm + ease_allowance_mm
```

Ease depends on garment category, age group, fit preference, fabric behavior and measurement type. Numbers must live in a versioned ease-profile table or configuration file, never scattered across route handlers.

## Transparent 100-point ranking

| Factor             | Maximum | Interpretation                                 |
|--------------------|---------|------------------------------------------------|
| Occasion           | 30      | Exact or compatible use-case match             |
| Climate and fabric | 20      | Breathability, warmth and fabric exclusions    |
| Fit                | 20      | Selected silhouette and supported ease profile |
| Style              | 15      | Overlap with selected style tags               |
| Colour             | 15      | Preferred palette and top-bottom compatibility |

## Determinism

Sort by total score descending, exact-match count descending and garment ID ascending. A fixed session, catalog version and rules version must always return the same result. Randomness is prohibited in the core ranking path.

## How the 100,000-person dataset is used

The live API must not scan every participant for each recommendation. A separate offline job should produce curated, de-identified summaries such as age-band percentiles or future trained artifacts. The online engine consumes a small, versioned rule or statistics package. Machine learning is considered only after a representative, consented and quality-reviewed dataset exists.

## SECTION 09

# Frontend journey and interface behavior

## Recommended route sequence

| Route          | Screen                   | Important behavior                                      |
|----------------|--------------------------|---------------------------------------------------------|
| /              | Introduction             | Explain outcome and measurement requirements            |
| /start         | Language and age gate    | Select consent branch before collecting personal data   |
| /consent       | Adult or guardian notice | Separate purposes and affirmative choices               |
| /garments      | Garment selection        | Show only supported combinations                        |
| /measurements  | Guided wizard            | Schema-driven sections, diagrams, attempts and autosave |
| /preferences   | Style choices            | Occasion, climate, cut, fit, colours and exclusions     |
| /review        | Submission review        | Show values, units, warnings and consent summary        |
| /results/:id   | Ranked outfits           | Top three, explanations and tailor specification        |
| /profile/:code | Data controls            | Correct, withdraw consent, export or delete             |

## Wizard requirements

- Group upper torso, arms and shoulders, lower body, advanced fields and review.
- Preserve values when navigating backward and after a recoverable server failure.
- Debounce draft autosave; use an idempotency key for each write.
- Display field-level server errors beside the related field, not as a generic toast only.
- Allow centimetres and inches, but never convert an already normalized value repeatedly.
- Support keyboard navigation, visible focus, screen-reader labels and mobile-width layouts.
- Provide plain-language and age-appropriate variants of instructions.

## Result card

- Local garment image, name, fabric, colour and silhouette.
- Total score and factor-by-factor explanation.
- Why the outfit matches the occasion, climate, fit and preferences.
- Body measurements used, calculated garment targets and any warnings.
- Rule, catalog and measurement-schema versions.
- Save, print and review-tailor-sheet actions.

## SECTION 10

# Backend modules and API contract

## Service boundaries

| Service                      | Responsibility                                                       |
|------------------------------|----------------------------------------------------------------------|
| ConsentService               | Age gate, purpose records, guardian state and withdrawal eligibility |
| MeasurementSchemaService     | Versioned field definitions and garment requirements                 |
| UnitConversionService        | Single conversion path into integer millimetres                      |
| MeasurementValidationService | Technical, cross-field, repeat and age-aware checks                  |
| QualityScoringService        | Completeness, agreement, provenance and review status                |
| FitCalculationService        | Select rule version and calculate finished targets                   |
| RecommendationService        | Filter, score, combine, rank and explain outfits                     |
| DatasetCurationService       | Consent filtering, pseudonymization and controlled exports           |

## Core endpoints

```
GET /health
GET /api/v1/measurement-schema
POST /api/v1/consents/start
POST /api/v1/consents/{id}/confirm
POST /api/v1/participants
POST /api/v1/measurement-sessions
PATCH /api/v1/measurement-sessions/{id}
POST /api/v1/measurement-sessions/{id}/measurements
POST /api/v1/measurement-sessions/{id}/validate
POST /api/v1/measurement-sessions/{id}/submit
POST /api/v1/recommendations
GET /api/v1/recommendations/{id}
POST /api/v1/participants/{id}/withdraw
DELETE /api/v1/participants/{id}/data
```

## API rules

- Use a stable /api/v1 prefix and structured error codes.
- Require Idempotency-Key on submission, consent confirmation and recommendation creation.
- Enforce request-size limits and reject unknown measurement codes.
- Never expose an unpaginated all-participants endpoint.
- Return rule, schema and catalog versions in recommendation responses.
- Keep route handlers thin; domain logic belongs in services and repositories.

## SECTION 11

# Local project structure and setup

---

## Repository layout

```
tailored-outfit/  
|-- README.md  
|-- Makefile  
|-- frontend/  
| |-- src/api/  
| |-- src/components/  
| |-- src/pages/  
| |-- src/features/consent/  
| |-- src/features/measurements/  
| |-- src/features/recommendations/  
| `-- vite.config.js  
`-- backend/  
    |-- app/api/  
    |-- app/core/  
    |-- app/models/  
    |-- app/schemas/  
    |-- app/repositories/  
    |-- app/services/  
    |-- app/data/  
    |-- scripts/  
    |-- tests/  
    `-- alembic/
```

## Local startup model

- Terminal 1 runs local PostgreSQL as a Fedora system service.
- Terminal 2 activates the Python virtual environment and runs Uvicorn with reload.
- Terminal 3 runs the Vite development server.
- Vite proxies /api and /health to http://127.0.0.1:8000, preventing most local CORS issues.
- A root Makefile standardizes install, migrate, seed, test and run commands.

## Environment configuration

```
APP_ENV=development  
DATABASE_URL=postgresql+psycopg://tailor_app:local_password@localhost:5432/tailor_db  
CONSENT_VERSION=1  
MEASUREMENT_SCHEMA_VERSION=1  
PROTOCOL_VERSION=1  
RULES_VERSION=1  
CATALOG_VERSION=1
```

Use .env.example with placeholders and keep the real .env out of version control. The initial system requires no API key and no GPU configuration.

## SECTION 12

# Testing, load simulation and quality gates

## Backend test matrix

| Area       | Required cases                                                            |
|------------|---------------------------------------------------------------------------|
| Units      | cm/inch conversion, round trip display, integer-mm normalization          |
| Validation | Required fields, bounds, NaN, infinity, duplicates, repeat tolerance      |
| Consent    | Purpose separation, guardian branch, withdrawal and deletion eligibility  |
| Database   | Constraints, rollback, migrations, cascade behavior and idempotency       |
| Fit        | Every garment/fit/fabric rule and version selection                       |
| Ranking    | Filtering, score arithmetic, compatible pairs and tie-breaking            |
| API        | Happy path, structured 422 errors, retries, pagination and payload limits |
| Curation   | Withdrawn and low-quality sessions excluded from research export          |

## Frontend test matrix

- Correct fields render for each garment selection.
- Age gate chooses the correct consent flow.
- Back and forward navigation preserves draft values.
- Unit changes preserve the physical measurement.
- Field-level API errors appear beside the field.
- Offline/retry behavior does not duplicate accepted submissions.
- Results expose reasons, warnings and specification values.
- Keyboard-only and mobile-width journeys complete successfully.

## Synthetic scale and load tests

Generate 100,000 participants with a fixed random seed, valid records, missing values, unusual but possible bodies, invalid values, disagreements between attempts and withdrawn consent. Insert in batches of 1,000.

| Target                 | Local quality gate                                                   |
|------------------------|----------------------------------------------------------------------|
| Integrity              | Zero duplicate final submissions and zero lost accepted measurements |
| Reliability            | Unexpected server errors below 0.1% in the test scenario             |
| Submission latency     | p95 below 300 ms at the agreed local concurrency                     |
| Recommendation latency | p95 below 500 ms for rules-based recommendations                     |
| Memory                 | No growth trend during sustained tests; exports remain streamed      |
| Recovery               | Backup can be restored and acceptance fixtures still pass            |

These are development targets on the known local machine, not production service-level guarantees. Record actual hardware, dataset size and concurrency with each benchmark.

**SECTION 13**

# Sampling, analytics and future model readiness

---

## Representative collection

A large dataset can still be biased. Define the population the product intends to serve, then monitor broad quotas across age band, region, garment-cut preference and measurement method. Do not assume equal counts automatically represent the target population.

- Track self-measured and professionally measured records separately.
- Monitor form abandonment and validation-failure rates by broad cohort.
- Review whether warnings disproportionately affect any cohort.
- Suppress very small aggregate cells to reduce re-identification risk.
- Document known sampling gaps rather than hiding them with synthetic balancing.

## Research-ready pipeline

- 1 Select only sessions with valid research consent.
- 2 Remove direct identifiers and generate dataset-specific random IDs.
- 3 Apply documented quality thresholds without deleting the source record.
- 4 Generalize region and retain age-at-measurement rather than date of birth.
- 5 Freeze schema, protocol and curation versions with each export.
- 6 Split future train, validation and test sets by participant, never by individual measurement row.

## When machine learning becomes justified

A model is considered only after there is enough representative, consented, reviewed data and a clearly defined prediction target. The baseline deterministic engine remains the comparison system. A future model must demonstrate improved fit outcomes, calibrated uncertainty and acceptable cohort-level performance before it can influence recommendations.

## Do not infer sensitive attributes

Do not train a model to infer sex, gender, ethnicity, disability or age from body measurements. Use voluntarily provided, purpose-limited fields only for the stated service or fairness analysis.

## SECTION 14

# Implementation roadmap

| Phase | Workstream                           | Required output                                                    |
|-------|--------------------------------------|--------------------------------------------------------------------|
| 0     | Data dictionary and product contract | Field purpose, units, codes, consent purposes, acceptance fixtures |
| 1     | End-to-end skeleton                  | React -> proxy -> FastAPI -> PostgreSQL health and test record     |
| 2     | Database foundation                  | Models, migrations, constraints, repositories, rollback tests      |
| 3     | Measurement schema                   | Dynamic API schema, conversion, validation and repeat attempts     |
| 4     | Adult vertical slice                 | One shirt flow from consent to deterministic result                |
| 5     | Catalog and recommendation engine    | Shirts, T-shirts, trousers, fit profiles and explanation scores    |
| 6     | Full React journey                   | Wizard, preferences, review, result cards and saved profiles       |
| 7     | Consent and data rights              | Purpose records, withdrawal, correction, export and deletion       |
| 8     | Guardian state machine               | Age gate, simulated verification and child-specific rules          |
| 9     | Synthetic scale                      | 100,000 participants, batching, profiling and index tuning         |
| 10    | Load and recovery                    | Locust scenarios, failure injection, backup and restore            |
| 11    | Pilot readiness review               | Accessibility, privacy, tailoring review and operational checklist |

## Build order principle

Finish a thin vertical slice before expanding breadth: choose shirt -> enter eight measurements -> submit -> receive one deterministic recommendation. Then add persistence, trousers, combination ranking, guardian flow and scale testing. This exposes integration failures early.

## Suggested solo-development cadence

| Block      | Focus                                                                |
|------------|----------------------------------------------------------------------|
| Days 1-2   | Contracts, repository, local PostgreSQL, migrations and health slice |
| Days 3-5   | Measurement schema, validators, attempts, tests and adult shirt flow |
| Days 6-8   | Catalog, ease rules, ranking, explanations and fixtures              |
| Days 9-12  | React wizard, preferences, results, errors and accessibility         |
| Days 13-15 | Consent, rights, guardian state and quality scoring                  |
| Days 16-18 | Synthetic generator, 100,000-record run, profiling and fixes         |
| Days 19-20 | Load, backup/restore, clean-install rehearsal and documentation      |

This is an engineering estimate, not a deadline guarantee. Tailoring-rule review, usability research and legal review are external dependencies and can extend the schedule.

## SECTION 15

# Risk register and mitigations

| Risk                   | Impact                   | Mitigation                                                                       | Gate                   |
|------------------------|--------------------------|----------------------------------------------------------------------------------|------------------------|
| Self-measurement error | Incorrect fit            | Diagrams, repeat attempts, tolerances, quality score and professional comparison | Pilot                  |
| Unvalidated ease rules | Misleading specification | Version rules, label provisional data and require apparel-specialist approval    | Before claims          |
| Child consent failure  | Legal and ethical harm   | Block collection until verified consent and jurisdictional review                | Before real child data |
| Duplicate submissions  | Biased dataset           | Idempotency key, uniqueness constraints and transaction tests                    | Before pilot           |
| Demographic conflation | Exclusion and poor fit   | Separate age, sex, gender and cut preference; measurements drive fit             | Schema review          |
| Sampling bias          | Weak generalization      | Quota dashboard, professional-measure subset and documented gaps                 | Each expansion         |
| Rule drift             | Irreproducible results   | Snapshot input, catalog, schema and rule versions                                | MVP                    |
| Over-engineering       | Delayed MVP              | Single service, one PostgreSQL database, no cloud or ML initially                | Continuous             |
| Deletion gaps          | Rights violation         | Data map, deletion tests, processor inventory and tombstone policy               | Before pilot           |
| Sensitive data in logs | Privacy breach           | Structured redaction and tests that inspect captured logs                        | MVP                    |

## Highest-risk assumptions

- The linked measurement guide is a useful field list, not sufficient evidence for universal pattern calculations.
- One set of adult rules cannot safely represent all children and growth stages.
- One hundred thousand records do not create scientific validity without reliable collection and representative sampling.
- A technically functioning guardian checkbox is not the same as legally valid verifiable parental consent.
- Local benchmark results will not predict production behavior without network, deployment and operational testing later.

**SECTION 16**

# Definition of done and launch gates

---

## Local engineering definition of done

- Fresh setup succeeds from README commands on Fedora.
- Database migrations can upgrade an empty database and a seeded database.
- The full adult browser journey succeeds without using Swagger manually.
- All supported garment selections request the correct fields.
- cm and inch entry normalize to identical physical values.
- Validation failures return field-specific, stable error codes.
- Recommendation output is deterministic and versioned.
- One hundred thousand synthetic participants load without duplicate final submissions.
- Tests cover withdrawal, deletion, retry and rollback paths.
- No external service or GPU is required.

## Before any real adult pilot

- Consent notice and retention schedule approved.
- Measurement instructions tested with non-developer users.
- Tailor reviews measurement definitions, tolerances and initial ease profiles.
- Database backup and restoration demonstrated.
- Correction, export and erasure journeys tested end to end.
- Incident contact and response procedure documented.

## Before any real child data

Do not proceed unless guardian-verification, consent, retention, child-friendly notice, access, correction, deletion, advertising restrictions and applicable jurisdictional requirements have been independently reviewed. The local simulated state machine is not sufficient authorization.

## Before expanding from 10,000 to 100,000

- Quality metrics remain acceptable by broad cohort.
- Sampling gaps and weighting plan are documented.
- The professional-measure comparison shows acceptable error levels.
- Load, retry, backup, restore, withdrawal and deletion tests all pass.
- Tailoring claims accurately reflect the validation evidence.
- Production hosting and security are handled in a separate reviewed plan.

**SECTION A**

# Appendix: API examples and local commands

---

## Example recommendation request

```
{
  "garments": ["shirt", "trousers"],
  "measurement_session_id": "<uuid>",
  "preferences": {
    "occasion": "office",
    "climate": "hot",
    "fit": "regular",
    "style": ["minimal", "classic"],
    "colours": ["navy", "white", "beige"],
    "avoid_fabrics": []
  }
}
```

## Frontend bootstrap

```
mkdir tailored-outfit && cd tailored-outfit
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install react-router-dom react-hook-form @tanstack/react-query
npm install -D vitest jsdom @testing-library/react @testing-library/jest-dom
```

## Backend bootstrap

```
mkdir -p backend && cd backend
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install "fastapi[standard]" sqlalchemy "psycopyg[binary]" alembic pydantic-settings pytest httpx faker
locust
pip freeze > requirements.txt
```

## Run and verify

```
# Backend terminal
source backend/.venv/bin/activate
uvicorn app.main:app --reload

# Frontend terminal
cd frontend
npm run dev

# Test
pytest -q
npm run test
```

## Synthetic scale command target

```
python -m scripts.generate_synthetic_data \
+ --participants 100000 \
+ --seed 42 \
+ --batch-size 1000
```

SECTION B

# Appendix: source references

| Source                                       | Used for                                                         | URL                                                                                                                                                                                                 |
|----------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tailoring Measurement Guide                  | Measurement list for shirts, trousers and T-shirts               | <a href="https://www.scribd.com/document/887767936/Key-Measurements">https://www.scribd.com/document/887767936/Key-Measurements</a>                                                                 |
| Vite - Getting Started                       | Current scaffolding and Node.js compatibility guidance           | <a href="https://vite.dev/guide/">https://vite.dev/guide/</a>                                                                                                                                       |
| FastAPI - First Steps                        | Local Uvicorn server and interactive API documentation           | <a href="https://fastapi.tiangolo.com/tutorial/first-steps/">https://fastapi.tiangolo.com/tutorial/first-steps/</a>                                                                                 |
| FastAPI - SQL Databases                      | Relational database integration guidance                         | <a href="https://fastapi.tiangolo.com/tutorial/sql-databases/">https://fastapi.tiangolo.com/tutorial/sql-databases/</a>                                                                             |
| SQLAlchemy documentation                     | ORM, sessions, dialects and transaction model                    | <a href="https://docs.sqlalchemy.org/">https://docs.sqlalchemy.org/</a>                                                                                                                             |
| PostgreSQL documentation                     | PostgreSQL data types and database behavior                      | <a href="https://www.postgresql.org/docs/current/">https://www.postgresql.org/docs/current/</a>                                                                                                     |
| Digital Personal Data Protection Act, 2023   | India: consent, rights and children's data provisions            | <a href="https://www.meity.gov.in/static/uploads/2024/06/2bf1f0e9f04e6fb4f8fef35e82c42aa5.pdf">https://www.meity.gov.in/static/uploads/2024/06/2bf1f0e9f04e6fb4f8fef35e82c42aa5.pdf</a>             |
| Digital Personal Data Protection Rules, 2025 | India: notices, safeguards and verifiable consent detail         | <a href="https://www.meity.gov.in/static/uploads/2025/11/53450e6e5dc0bfa85ebd78686cadad39.pdf">https://www.meity.gov.in/static/uploads/2025/11/53450e6e5dc0bfa85ebd78686cadad39.pdf</a>             |
| FTC COPPA guidance                           | US guidance on verifiable parental consent for children under 13 | <a href="https://www.ftc.gov/business-guidance/resources/complying-coppa-frequently-asked-questions">https://www.ftc.gov/business-guidance/resources/complying-coppa-frequently-asked-questions</a> |

Source note: software requirements and privacy rules can change. Re-check official documentation immediately before implementation milestones that depend on a version number or before collecting real personal data.

## Final recommendation

Begin with the adult shirt vertical slice and build the measurement schema, database constraints and deterministic rules before polishing the full catalog. Use 100,000 synthetic records to validate engineering scale. Treat tailoring accuracy, representative sampling and real child-data consent as independent approval gates, not assumptions.